

Towards Zero Rotation and Beyond: Architecting Neural Networks for Fast Secure Inference with Homomorphic Encryption

Yifei Cai	Yizhou Feng	Qiao Zhang	Chunsheng Xin	Hongyi Wu
Iowa State University	Old Dominion University	Shandong University	Iowa State University	University of Arizona
Ames, IA	Norfolk, VA	Qingdao, Shandong	Ames, IA	Tucson, AZ
yifeic@iastate.edu	yfeng002@odu.edu	qiao.zhang@sdu.edu.cn	cxin@iastate.edu	mhwu@arizona.edu

Abstract—Privacy-preserving deep learning addresses privacy concerns in Machine Learning as a Service (MLaaS) using Homomorphic Encryption (HE) for linear computations. Nevertheless, the high computational cost remains a challenge. While prior work has attempted to improve the efficiency, most are built upon models originally designed for plaintext inference. These models are inherently limited by architectural inefficiencies when adapted to HE settings. We argue that substantial efficiency improvements can be achieved by designing networks specifically tailored to the unique computational characteristics of HE, rather than retrofitting existing plaintext models. Our design comprises two main components: the building block and the overall architecture. The first, StriaBlock, targets the most expensive HE operation—Rotation. It integrates ExRot-Free Convolution and a novel Cross Kernel, completely eliminating the need for external Rotation and requiring only 19% of the internal Rotation operations compared to plaintext models. The second component, the architectural principle, includes the Focused Constraint Principle, which limits cost-sensitive factors while preserving flexibility in others, and the Channel Packing-Aware Scaling Principle, which dynamically adapts bottleneck ratios based on ciphertext channel capacity that varies with network depth. These strategies efficiently control the local and overall HE cost, enabling a balanced architecture for HE settings. The resulting network, StriaNet, is comprehensively evaluated. While prior works primarily focus on small-scale datasets such as CIFAR-10, we conduct an extensive evaluation of StriaNet across datasets of varying scales, including large-scale (ImageNet), medium-scale (Tiny ImageNet), and small-scale (CIFAR-10) benchmarks. At comparable accuracy levels, StriaNet achieves speedups of $9.78\times$, $6.01\times$, and $9.24\times$ on ImageNet, Tiny ImageNet, and CIFAR-10, respectively.

Index Terms—Privacy Preserving Machine Learning, Homomorphic Encryption, Model Architecture.

I. INTRODUCTION

Deep learning (DL) has become a ubiquitous tool, significantly enhancing productivity and solving complex problems with profound societal implications in the present era. However, the construction of a DL model necessitates access to vast datasets, substantial computational capabilities, and specialized expertise. That is a formidable obstacle often beyond the reach of many users and organizations. In response to this challenge, the concept of Machine Learning as a Service (MLaaS) has emerged as a pragmatic solution [1]. MLaaS

empowers service providers equipped with ample resources to develop well-trained, advanced DL models, offering them as a service accessible to a wide range of users.

However, privacy emerges as a paramount concern in MLaaS, where clients are reluctant to share their valuable private data with the server, and the server guards its model parameters as valuable intellectual property. To mitigate this privacy dilemma, privacy-preserving MLaaS (PP MLaaS) integrates cryptographic primitives into the computation process of DL models, as demonstrated in several recent frameworks, such as Cryptonets [2], SecureML [3], MiniONN [4], GAZELLE [5], DELPHI [6], CrypTFlow2 [7], Cheetah [8], Iron [9], BOLT [10] and BumbleBee [11]. Commonly adopted cryptographic primitives in these frameworks include Homomorphic Encryption (HE) [12], Garbled Circuits (GC) [13], Oblivious Transfer (OT) [14], and Secret Sharing (SS) [15].

Fully Homomorphic Encryption (FHE) [16], [17] enables arbitrary computations on encrypted data without decryption or private key access, while Partially Homomorphic Encryption (PHE) supports a limited set of functions, such as additions or depth-bounded operations. PHE is simpler and faster than FHE but less versatile. FHE is often used in scenarios with minimal server-client interaction, requiring nonlinear functions to be approximated as polynomials and relying on computationally expensive bootstrapping to manage noise [18]. SS schemes, which involve multiple parties, are commonly applied in settings such as federated learning and distributed computation. OT is typically used to securely exchange messages between parties and facilitate the secure evaluation of nonlinear operations [14].

Efficiency Challenges for PP MLaaS: Given that DL models consist of both linear and nonlinear functions, some frameworks adopt hybrid cryptographic primitives that utilize HE for linear computations and Multi-Party Computation (MPC) protocols—such as GC and OT—for nonlinear operations. State-of-the-art examples include HE-GC-based frameworks like GAZELLE and DELPHI, as well as HE-OT-based frameworks such as CrypTFlow2 and Cheetah. Alternatively, some frameworks [18], [19] approximate nonlinear functions, such

as ReLU, using polynomials, thereby enabling their evaluation within the HE domain as part of linear computation. Overall, HE plays a pivotal role in PP MLaaS frameworks, which motivates this work to focus specifically on HE. Many real-time applications demand rapid response times [20]; however, the use of cryptographic primitives such as HE introduces significant computational overhead, limiting the practicality of current solutions. For example, performing inference on a single ImageNet image using ResNet-50 [21] within state-of-the-art privacy-preserving frameworks [5]–[8], [22] typically takes approximately 100 to 400 seconds, rendering such latency infeasible for real-world deployment.

Existing Efficiency Enhancement Approaches and Limitations: To enhance the efficiency of PP MLaaS, several studies have focused on optimizing inference under HE. Some works, such as SpENCNN [23] and HyPHEN [19], explore HE-aware data packing schemes to better exploit SIMD parallelism, achieving approximately $1.85\times$ speedup. Cheon et al. [24] introduce a channel-by-channel convolution approach whose efficiency scales with batch size, reaching up to $5.2\times$ speedup with a large batch size of 512. Similarly, HeLayers [25] adopts HE-friendly data packing combined with tailored convolution operations, yielding a $1.2\times$ speedup and further improvements as batch size increases. Other approaches, such as AutoFHE [18] and REDsec [26], apply function-level optimizations, including polynomial approximations and discretized network representations, to accelerate inference under HE. However, these methods may result in trade-offs in model accuracy. Another representative work, Cheetah [8], adopts specialized encoding schemes and polynomial multiplication to avoid the high cost from HE Rotation. In addition, techniques such as MOSAIC [27] and the work by Legiest et al. [28] emphasize model compression through pruning and quantization to reduce the complexity of the original model while preserving inference accuracy.

Despite these advancements, existing methods represent optimizations built upon original models and are inherently constrained by architectures designed for plaintext inference. Models such as VGG and ResNet were originally developed to minimize computational costs in plaintext settings, which can be inefficient—or even detrimental—when applied to HE scenarios. For instance, the pruning-based method MOSAIC [27] reduces the inference time of VGG-16 on CIFAR-10 from 119.72s to 41.69s, achieving a $2.87\times$ speedup (see Figure 1). While this performance gain is notable, substantial room for improvement remains. The core issue is that the baseline model—VGG—is excessively costly in ciphertext settings, and pruning faces inherent limitations, as it must preserve essential structural components to maintain accuracy. *But what if the core structures of plaintext models are themselves inherently inefficient for HE?* In such cases, the effectiveness of any optimization is ultimately bounded by the plaintext-oriented design of the original model architecture.

Reevaluating Plaintext Model Efficiency Under HE: For plaintext DL inference, several classic backbone neural net-

TABLE I
EFFICIENCY REVERSAL: MODELS EFFICIENT IN PLAINTEXT BECOME INEFFICIENT UNDER HE.

Model Pair	DenseNet-169 vs ResNet-50	ResNet-101 vs VGG-11	MobileNet(1.4) vs ResNet-18
Plaintext Settings: FLOPs ($\times 10^9$)	7.07 < 7.45	14.86 < 19.54	1.23 < 3.59
HE Settings: HE Cost (Sec)	397.07 > 268.76	540.09 > 311.43	122.88 > 125.53
Relative Cost Ratio (Plaintext $\rightarrow$ HE Settings)	95% $\rightarrow$ 148% Reversed	76% $\rightarrow$ 173% Reversed	34% $\rightarrow$ 98% Advantage Lost

works are widely used, including VGG [29], ResNet [21], DenseNet [30], and MobileNet [31]. These architectures incorporate various optimizations to improve efficiency in plaintext inference, such as advanced downsampling strategies, dimensionality reduction, channel pruning, and structurally efficient designs. Similar optimization techniques are also evident in many other related works [32]–[36]. As shown in Table I, we evaluate the inference cost divergence of these models under plaintext and HE settings using two metrics: *FLOPs* and *HE cost*. FLOPs (floating point operations) is a standard metric for quantifying the computational cost of plaintext inference. For ciphertext inference under HE, we adopt an aggregated HE cost metric that unifies the overall cost of different HE operations. Table I presents three comparison pairs, illustrating that certain architectures exhibit advantages in the plaintext setting but not under HE inference. First, consider DenseNet. Owing to its refined dimensional adjustments, DenseNet outperforms ResNet in terms of FLOPs. However, this advantage does not hold in HE settings. Its architecture leads to increased HE cost, reversing the efficiency ranking between ResNet and DenseNet. Second, we observe an amplification effect under HE. Although ResNet achieves substantially lower FLOPs than VGG—even ResNet-101 is more efficient than VGG-11—this advantage diminishes in the HE context. Some structural elements that are beneficial for plaintext inference introduce disproportionately high costs when executed under HE, amplifying inefficiencies. A similar trend is evident in the comparison between MobileNet and ResNet. These observations underscore the importance of analyzing the distinct cost distribution characteristics of HE. To develop truly HE-efficient architectures, it is crucial to design networks that explicitly consider and minimize HE cost.

Our contributions:

This work addresses a critical gap in current research—the lack of neural network architectures explicitly designed for HE settings. We contend that significant efficiency gains can be achieved by developing models specifically tailored to the computational constraints and unique characteristics of HE, rather than repurposing architectures originally designed for plaintext inference. The main contributions of this work are summarized as follows:

(I) Firstly, we design the building block. Through analysis, we find that among the three fundamental HE operations—Rotation, Multiplication, and Addition—Rotation, which includes both in-Rot and ex-Rot operations, is the most computationally expensive due to its reliance on the costly key switching operation [5], [7], [8]. Unlike in plaintext networks, where computational costs are more evenly distributed across

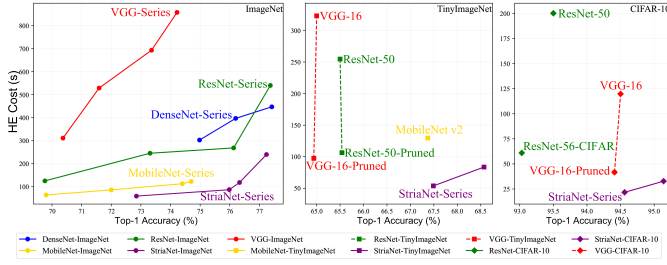


Fig. 1. Performance of StriaNet across ImageNet, Tiny ImageNet and CIFAR-10 datasets.

layers and closely aligned with model parameter counts, the cost of HE operations—particularly Rotation—is heavily concentrated within specific model structures in ciphertext settings. To address this challenge, we propose the **StriaBlock**, a novel building block specifically designed to avoid high-cost HE structures and thereby achieve intrinsic HE efficiency. StriaBlock significantly reduces, or even eliminates, both in-Rot and ex-Rot operations within convolutional layers, thereby minimizing the computational overhead associated with Rotation. It incorporates two key design elements:

- **ExRot-Free Pattern:** We introduce the exRot-Free pattern, an extremely HE-Efficient convolution layer pattern that eliminates the need for the ex-Rot operation.
- **Cross Kernel:** We introduce the Cross Kernel, maintaining comparable representational capacity while requiring only 19% of the in-Rot operations compared to traditional kernel.

(II) Secondly, we establish an architectural mechanism aimed at minimizing rotation operations as well. This mechanism is guided by two key principles. First, to minimize the Rotation cost at a specific model layer, it is essential to identify the dominant contributing factor. In plaintext settings, the computational cost of a layer is typically influenced by multiple factors—such as the input and output dimensions—relatively equally. However, in ciphertext settings, the dominant HE cost is often governed by a single critical factor. For example, in certain layers, the primary HE cost from in-Rot is predominantly determined by the size of the input channel dimension (see discussion in Sec. III-B). Thus, our first principle is:

- **Focused Constraint Principle:** *Constrain cost-sensitive factor while allowing other parameters to vary freely, improving overall network performance without incurring excessive cost.*

Moreover, during local architectural adjustments, we observe that certain layers are highly sensitive to changes in specific dimensions—small increases can trigger sharp surges in HE cost. In contrast, other layers display high tolerance, permitting significant dimensional expansion with minimal cost impact. We investigate these behavioral patterns and identify HE cost fluctuations as a function of network depth. In HE settings, network depth influences the ciphertext’s channel packing capacity thus affects the complexity of HE operations. Building on these observations, we extend our local adjustment strategy into a global optimization approach across the entire network and propose the second principle:

- **Channel Packing-Aware Scaling Principle:** *Dynamically*



Fig. 2. Privacy-preserving MLaaS.

adjust the bottleneck ratio of each building block based on ciphertext channel packing capacity, which varies with network depth. This strategy effectively regulates the overall HE cost and supports the development of a balanced, efficient architecture.

Following these architectural principles, we construct the network using the proposed building blocks, resulting in **StriaNet**—a neural network and corresponding design methodology for privacy-preserving deep learning.

(III) To evaluate the performance of StriaNet, comprehensive experiments are conducted. The existing works [18], [19], [23]–[28] have only considered small-scale neural networks and datasets, such as CIFAR-10. To compare, we perform an extensive evaluation of StriaNet across datasets of varying scales, including large-scale (**ImageNet**), medium-scale (**Tiny ImageNet**), and small-scale (**CIFAR-10**) benchmarks. As shown in Figure 1, StriaNet achieves excellent performance. At comparable accuracy levels, it delivers speedups of $9.78\times$, $6.01\times$, and $9.24\times$ on ImageNet, Tiny ImageNet, and CIFAR-10, respectively. The models and codes are available at: <https://github.com/caiyifei2008/StriaNet>.

(IV) Our HE-Efficient architecture offers a potential solution to the communication challenges identified in Cheetah [8]. Specifically, our exRot-Free pattern architecture minimizes HE computational cost without introducing additional communication overhead. Compared to Cheetah, StriaNet maintains the same computational complexity while significantly optimizing the communication of the target convolutional layer—achieving a $31.4\times$ reduction. This leads to a $13.1\times$ speedup for the target layer and a $1.73\times$ overall speedup in end-to-end inference.

The remainder of this paper is organized as follows: Sec. II introduces the system model, threat model, security analysis, and packed HE. Sec. III proposes StriaNet, a highly HE-Efficient DL Network and architecting methodology, including a novel building block-StriaBlock and the HE-Efficient architecture Principles. In Sec. IV, we discuss the experimental results. Sec. V concludes the paper.

II. PRELIMINARIES

A. System Model

In this study, we focus on PP MLaaS, depicted in Figure 2. It involves an interaction between two entities: the client (C) with sensitive data, exemplified by medical records, and the server (S) with a high-performance DL model capable of generating prediction. Privacy concerns arise during their collaborative exchange, with the client adamant about preventing any disclosure of its private data, including to the server. Simultaneously,

the server is reluctant to disclose model parameters due to substantial training resources. Consequently, the objective of PP MLaaS is to protect the client’s input from the server while maintaining the opacity of the server’s model parameters.

Aligned with prior contributions in the domain of PP MLaaS [2]–[8], [22], [37]–[52], our focus centers on deep CNNs, a pivotal and highly successful class of DL models extensively employed across diverse applications [29], [53], [54]. This study aims to improve the computational efficiency of linear operations, which comprise over 90% of the total inference cost in DL models [5], [7]. Within CNNs, two core linear operations—convolution and dot product—are critical, with convolution being particularly dominant in terms of computational cost during inference. Experimental evidence demonstrates the significant cost contribution of convolution in PP MLaaS; for instance, it accounts for approximately 99.05% of the linear computation in ResNet-50 [7], [8], [25]. Convolution involves convolving the input feature map with the kernels of the convolutional layer. In the context of MLaaS, the kernels is situated on the server, whereas the input feature is acquired from the client. Meanwhile, we follow established strategies for privacy-preserving nonlinear operations, such as max-pooling and ReLU activation, in accordance with state-of-the-art frameworks [5], [7], [8], [18], [19].

B. Threat Model and Security Analysis

We adhere to the semi-honest adversary model, a model widely employed in state-of-the-art PP MLaaS frameworks like GAZELLE [5], DELPHI [6], CrypTFlow2 [7], Cheetah [8], and MiniONN [4]. Within this model, both the client and the server conscientiously follow the protocol while attempting to infer additional information from the exchanged messages. Following the same definition of the cryptographic inference protocol in DELPHI [6], the server holds a model $\mathcal{W}$ consisting of N layers $W_1, W_2, \dots, W_N$, and the client holds the input vector x . During the PP MLaaS interactions, before sending it to the server, the input vector x is encrypted into ciphertext $\mathbf{x}$. On the server side, linear computations, such as convolutions, can be represented as $\mathbf{y} = F(W_i, \mathbf{x})$ where $i \in \{1, 2, \dots, N\}$. The security of this process relies on the semantic security of homomorphic encryption algorithms, such as BFV [12] and CKKS [55]. StriaNet, in particular, is a highly HE-Efficient DL network architecture that requires fewer HE operations for the same model capacity. From a secure inference standpoint, StriaNet maintains the same level of security, as it introduces no additional computational modules and does not rely on new algorithms. StriaNet is compatible with existing PP MLaaS frameworks and optimizations, making it a strong baseline backbone model for HE-based scenarios. It maximizes the efficiency of these optimizations by offering a better baseline. Overall, under the semi-honest threat model, StriaNet is considered secure.

C. Packed HE

Homomorphic Encryption (HE) is recognized as a significant breakthrough in the cryptographic domain due to its ability to facilitate linear computations between ciphertext

without the need for decryption [56]. This capability enables computational results to be directly calculated in encrypted form, preserving confidentiality. HE plays a pivotal role in the domain of PP MLaaS, where the client can encrypt sensitive data and send ciphertext to the server. Subsequently, the server can perform computations directly over the ciphertext using its DL model, obtaining prediction results without compromising access to the client’s confidential information [1], [57].

In plaintext deep learning, linear functions are accomplished through common multiply-add operations. However, in the HE-based scenario [5], [7], all linear operations, including convolution in convolutional layers and dot product operations in FC layers, are realized through the integration of three fundamental HE operations:

- **Homomorphic Addition (Add, “ $\oplus$ ”):** Consider two plaintext vectors, denoted as $\mathbf{v}_1$ and $\mathbf{v}_2$, which are respectively packed and encrypted into $[\mathbf{v}_1]_c$ and $[\mathbf{v}_2]_c$. Here, $[\cdot]_c$ signifies ciphertext encrypted by the client. The Add operation between ciphertext $[\mathbf{v}_1]_c$ and $[\mathbf{v}_2]_c$ mirrors common plaintext addition but operates element-wise, yielding the sum as ciphertext $[\mathbf{v}_1 \oplus \mathbf{v}_2]_c$.
- **Homomorphic Multiplication (Mult, “ $\odot$ ”):** The Mult operation between ciphertext $[\mathbf{v}_1]_c$ and plaintext vector $\mathbf{v}_2$ is executed through element-wise multiplication, resulting in the product ciphertext $[\mathbf{v}_1 \odot \mathbf{v}_2]_c$. It should be noted that, in typical CNN inference, there is no scenario where a Mult operation occurs between two ciphertexts.
- **Homomorphic Rotation (Rot):** The rotation operation performs a cyclic shift of the elements within a ciphertext. Given a ciphertext $[\mathbf{v}]_c$, rotating it by i positions entails shifting all elements in a cyclic manner by i positions in reverse order. This results in a new ciphertext, denoted as $[\mathbf{v}(+i)]_c$. Among the three fundamental HE operations, rotation is the most computationally expensive, as it involves costly key switching procedures [5], [7], [8].

It’s essential to note that both the Add and Mult operations function in an **element-wise manner**. Thus, directly summing up the values of a vector is not feasible, as these elements reside in different slots. The **Rotation** operation addresses this challenge by rotating the ciphertext, aligning internal elements for subsequent operations.

Within PP MLaaS frameworks, it is common practice to encrypt and aggregate multiple input channels into a single ciphertext to enhance efficiency [44], [58], [59]. Packed HE significantly boosts performance by enabling SIMD-style parallelism, where a single ciphertext encapsulates multiple plaintext values. This allows a single operation to process many data points simultaneously, resulting in more than an order-of-magnitude speedup [5], [7], [8]. Notably, the size of input features typically diminishes as one progresses from the initial layer to deeper layers within a DL model due to the down-sampling. For instance, in experiments with the ResNet on the ImageNet dataset, the input size reduces from 224×224 at the first layer to 7×7 at the deep layer. To ensure both safety and efficiency, the length of a single

ciphertext is conventionally established at 8192 slots in PP MLaaS frameworks [5], [7], [44], [58], [59]. As a result, the number of input channels that can be packed within a single ciphertext—referred to as the **channel packing capacity** and denoted by c_n —increases significantly, from 2 in shallow layers to 128 in deeper layers. The notation used in this paper is as follows:

Notation	Definition	Notation	Definition
c_i	input channels	c_o	output channels
k_w	width of the kernel	k_h	height of the kernel
W	width of the input	H	height of the input
$\mathbb{N}_x$	the set $\{0, 1, \dots, x-1\}$	$\lceil x \rceil$	ceiling operation on x
c_n	channel packing capacity: number of packed channels in a ciphertext		

III. STRIANET: HE-EFFICIENT DEEP LEARNING NETWORK AND ARCHITECTING METHODOLOGY

To address a critical gap in current research, the lack of neural network architectures explicitly designed for HE settings, we introduce StriaNet, a neural network and accompanying architectural methodology that fully considers the unique computational characteristics of HE-based convolution. StriaNet demonstrates significantly higher HE efficiency compared to existing models originally designed for plaintext inference.

As with all network architecture designs, StriaNet consists of two key components: the **building block design** and the **network architectural principle**. The first component, StriaBlock, incorporates ExRot-Free Convolution and a novel Cross Kernel, eliminating the need for ex-Rot and requiring only 19% of the in-Rot operations compared to plaintext models. The second component, the architectural principle, includes Focused Constraint Principle, which limits cost-sensitive factors while preserving flexibility in others, and Channel Packing-Aware Scaling Principle, dynamically adapts bottleneck ratios based on ciphertext channel capacity that varies with network depth. These strategies efficiently control the local and overall HE cost, enabling a balanced architecture for HE settings.

A. Building Block Design

1) **exRot-Free Pattern in MIMO**: The general convolution computation with Multiple Input Multiple Output (MIMO) is a configuration commonly used in many DL models. Figure 3 illustrates the MIMO scheme¹ when $c_n = 2$. In this case, 4 input channels, denoted as c_1, c_2, c_3, c_4 , are packed and encrypted into 2 ciphertext, $[v_1]_C$ and $[v_2]_C$. These ciphertext are then convolved with 4 filters, corresponding to the 4 input channels. Consequently, the output is obtained, consisting of two encrypted ciphertexts, namely Output-1 and Output-2. To compute the Output, we first perform a convolution (denoted by the $*$ symbol) between the input ciphertext $[v_1]_C$ and the kernel group $\{k_{11}, k_{22}\}$, resulting in the ciphertext $[c_1 k_{11}, c_2 k_{22}]_C$, referred to as the *intermediate ciphertext*.

¹For conciseness, we present the Grouped Output Rotation MIMO scheme, which serves as the default approach in PP MLaaS frameworks [7]. For details on an alternative, the Input Rotation MIMO, please refer to Appendix-A. Note that the design proposed in this work is applicable to both the Output Rotation and Input Rotation MIMO schemes.

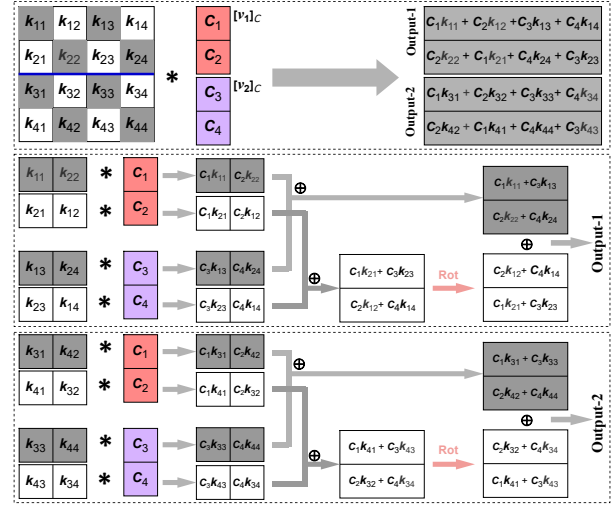


Fig. 3. Grouped Out-Rot MIMO ($c_i = c_o = 4, c_n = 2$).

Next, the same input ciphertext $[v_1]_C$ is convolved with the kernel group $\{k_{21}, k_{12}\}$ to produce another intermediate ciphertext $[c_1 k_{21}, c_2 k_{12}]_C$. These intermediate ciphertexts are subsequently combined through a series of additions using the HE Add operation to generate the final Output. To conform with the element-wise computation manner of Packed HE, several HE Rotation operations are applied to realign the intermediate ciphertexts. In the MIMO configuration, this Rotation operation is referred to as external Rotation, abbreviated as **ex-Rot**. As discussed in Sec.II-C, in HE settings, there are three fundamental HE operations: Rotation, Mult and Add. And among these, Rotation is by far the most computationally expensive operation [5], [7], [8]. Moreover, based on our analysis, such as ResNet-50 running on the ImageNet dataset, we find that ex-Rot account for a substantial 82.09% of the total Rotation cost. This observation strongly motivates our design focus: *to prioritize the reduction, and ideally the elimination, of ex-Rot operations.*

exRot-Free Pattern

For the kernel with kernel matrix index $[I_{row}, I_{col}]$:

$$I_{row} = k + c_n \times i \quad \text{s.t.} \quad I_{row} \in \mathbb{N}_{c_o},$$

$$I_{col} = k + c_n \times j \quad \text{s.t.} \quad I_{col} \in \mathbb{N}_{c_i},$$

where $k \in \mathbb{N}_{c_n}$; $i \in \mathbb{N}_m \leftarrow m = \lceil \frac{c_o}{c_n} \rceil$; $j \in \mathbb{N}_n \leftarrow n = \lceil \frac{c_i}{c_n} \rceil$.

Upon deep analysis of the MIMO schemes, we observe that not all convolutional layer structures necessitate ex-Rot. Specifically, only those structures involving intermediate ciphertexts whose channel order differs from that of the original input require ex-Rot for reordering. Therefore, the need for ex-Rot can be eliminated if the convolutional layer is constructed exclusively using structures (i.e., convolution kernels) associated with intermediate ciphertexts whose channel order aligns with the original input. Building on this insight, we design a universally **exRot-Free pattern**. This pattern is characterized by the main diagonal of each $c_n \times c_n$ kernel matrix, which is visually highlighted in grey in Figure 3. Kernels adhering to this exRot-Free pattern do not require any ex-Rot operations. Moreover, layers composed entirely of this structure also eliminate the need for ex-Rot during the MIMO process. (Note

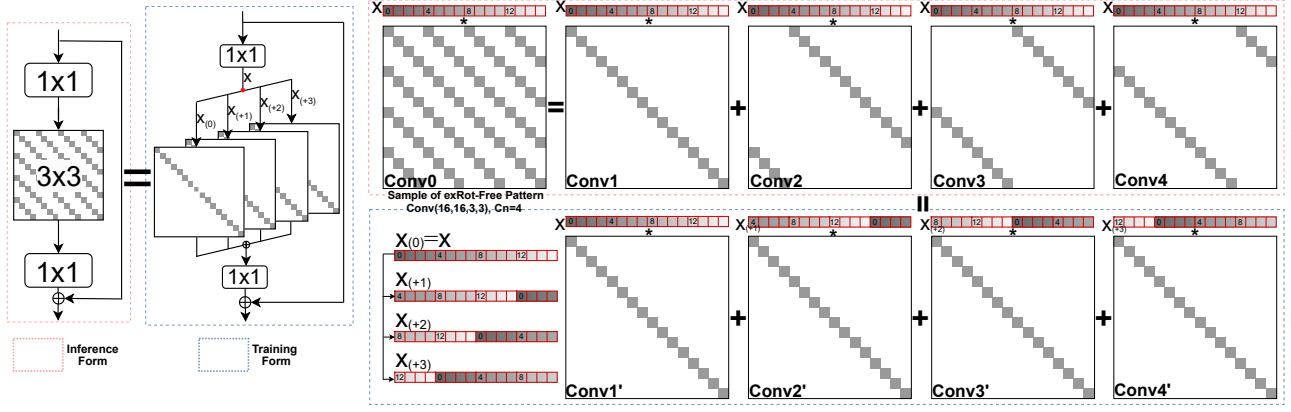


Fig. 4. StriaBlock with exRot-Free Pattern ($c_i = c_o = 16, c_n = 4$).

that c_n , the channel packing capacity, is one central element that permeates our design and will be further discussed in subsequent sections.)

Figure 4 illustrates the building block that adopts the exRot-free pattern, exhibiting a distinctive striated texture. We name this block **StriaBlock**. StriaBlock is designed as an inverted residual block composed of a sequence of three convolutional layers with kernel sizes of 1×1 , 3×3 , and 1×1 , respectively. The 3×3 layer serves as the primary feature extractor, while the two 1×1 layers are responsible for channel scaling and mixing. As shown in Figure 4, the 3×3 layer adopts the exRot-Free pattern, which brings two major benefits.

First, from the perspective of **computational efficiency**, the 3×3 layer is typically the most computation-intensive part of the block, particularly due to its heavy reliance on ex-Rot operations. Applying the exRot-Free pattern to this layer maximizes the efficiency gain, reducing the overall HE cost by up to 60%.

Second, in terms of **network capacity**, a layer following the exRot-Free pattern can, in principle, be decomposed into multiple simple depthwise convolutional layers, each operating on input data with a different channel order. As illustrated on the right side of Figure 4, the top row shows a convolutional layer (Conv0) following the exRot-Free pattern convolving with the input $x_{(0)}$. This corresponds to the inference form. Conv0 can be deconstructed into four sub-layers (Conv1 to Conv4), each also convolving with the original input $x_{(0)}$. These sub-layers can then be transformed by reordering their input channels (i.e., columns of the kernel matrix) by 4, 8, and 12 channels, resulting in Conv1' to Conv4'. Simultaneously, the input $x_{(0)}$ is reordered accordingly into $x_{(+1)}$, $x_{(+2)}$, and $x_{(+3)}$, forming the training form. Importantly, the training and inference forms are functionally equivalent. During training on the server side, all operations are conducted in plaintext, making channel reordering straightforward. After training, the model is reassembled into Conv0 with the original input $x_{(0)}$, requiring no HE Rotation operation during encrypted inference.

Functionally, the exRot-Free pattern behaves like multiple parallel depthwise convolutions on inputs with varied channel orders. This structure enables effective extraction of diverse

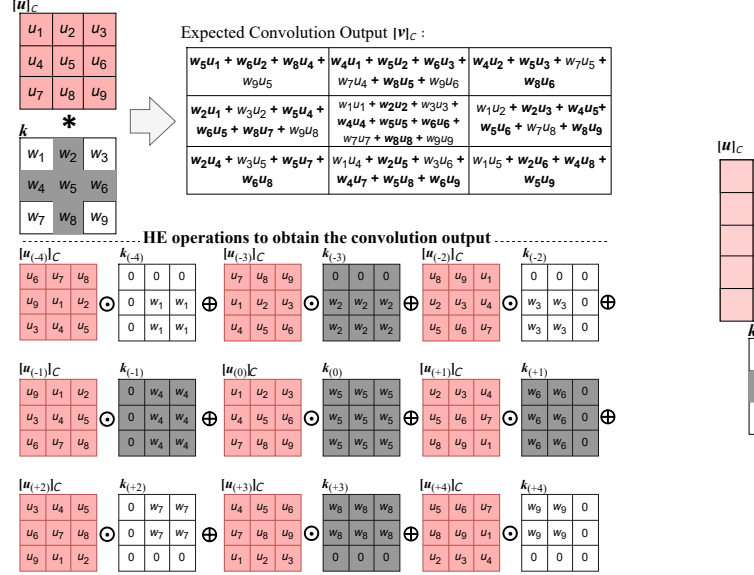


Fig. 5. SISO ($W = H = 3, k_w = k_h = 3$).

features from the input, which are then integrated and preserved by the subsequent 1×1 layer. As a result, StriaBlock offers both high HE efficiency and strong representational capacity, making it a powerful and practical building block for secure inference with HE.

2) **Cross Kernel in SISO**: The proposed exRot-Free pattern eliminates the need for ex-Rot operations. Furthermore, we continue to address the other HE Rotation operation, in-Rot, which is required during the inner kernel computation, the convolution within Single Input and Single Output (SISO).

In Figure 5, the client encrypts its input u (with the size of $W \times H$) as $[u]_C$. This ciphertext is then sent to the server to convolve with the corresponding plaintext kernels k (with the size $k_w \times k_h$, and the $*$ in the figure denotes the convolution operator). The encrypted convolution output $[v]_C$ is obtained. Specifically, the convolution operation is achieved by first placing the kernel at each location of the input feature and then summing up all the element-wise products between the input values and the corresponding kernel values covering the respective positions. However, as previously discussed, the server cannot directly perform the summation because, in the HE-based setting, the Add operation functions in an element-

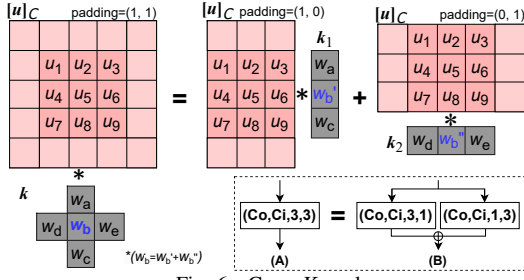


Fig. 6. Cross Kernel.

wise manner. Specifically, the Add operation can only sum values located at the same slot across multiple ciphertexts; it cannot aggregate values from different slots within a single ciphertext. To address this limitation, the Rotation operation is employed to shift the ciphertext, aligning the desired slots for correct summation.

As illustrated in Figure 5, to conform with the element-wise computation manner, the original inputs $[u_{(0)}]_C$ must be rotated to $[u_{(-4)}]_C \sim [u_{(+4)}]_C$ using eight in-Rot operations. This introduces a significant computational cost, especially considering the large-scale input channels in the overall network. Even with the use of hoisting techniques [5], which can improve the batch efficiency of such operations, the cost remains substantial. According to our analysis, not all parameters within a kernel incur the same cost. For instance, in a 3×3 kernel, every parameter except the central one, k_5 , requires an in-Rot operation. However, the cost associated with these operations can vary significantly. In typical settings, for efficiency and security, the length of the HE ciphertext is configured as a power of two (e.g., 8192), and the input dimensions are padded accordingly to a power-of-two size. Let us denote the padded input dimension as N . The required in-Rot operations for the kernel weights involve position shifts of ± 1 , $\pm N$, and $\pm N \pm 1$. Due to the inherent properties of HE ciphertexts, rotations by positions that are powers of two are generally more efficient [58]–[60]. Consequently, in-Rot operations with position shifts of $\pm N$ benefit from this efficiency, and those with small shifts such as ± 1 also incur relatively low overhead. In contrast, in-Rot operations by positions $\pm N \pm 1$ are considerably more expensive.

TABLE II
KERNEL IN-ROT COST ANALYSIS.

Input dimension N	4	8	16	32	64
Cross kernel in-Rot (ms)	23.27	22.78	23.8	23.93	23.94
Regular kernel in-Rot (ms)	70.58	77.41	91.85	105.00	128.85
Relative cost (%)	33%	29%	26%	23%	19%

We refer to these high-cost model weights as corner weights, as they are typically located at the four corners of a kernel. Then we propose the **Cross Kernel**, as illustrated in Figure 6. As the name suggests, the Cross Kernel retains only the five weights along the cross-shaped backbone of the kernel, thereby excluding all expensive corner weights. As shown in Table II, the in-Rot cost of the Cross Kernel is significantly lower than that of the regular kernel, and this advantage becomes increasingly pronounced as N grows. Specifically, when $N = 64$, using the Cross Kernel reduces the in-Rot cost to as low as 19% of that incurred by a regular kernel. Importantly, the

Cross Kernel fundamentally differs from pruning methods, where model weights are removed post-training. Such pruning typically introduces convergence challenges, including accuracy degradation, and necessitates iterative pruning and fine-tuning, which are both time-consuming and computationally expensive. In contrast, the Cross Kernel is designed without corner weights from the outset, thus avoiding these issues entirely.

This design is achieved through the degenerated Spatially Separable Convolution [61]. As shown in Figure 6, the convolution between the input and a Cross Kernel can be equivalent to the sum of two convolutions between the input and two sub-kernels with dimensions $(3, 1)$ and $(1, 3)$ separately. Thus, one convolutional layer with the Cross Kernel is equivalent to two parallel convolutional layers with $(3, 1)$ and $(1, 3)$ kernels, respectively. More generally, for an arbitrary $M \times N$ kernel (where M and N are either odd or adjusted to be odd), the Cross Kernel includes only the weights along the cross backbone—that is, those in the $\frac{M+1}{2}$ -th row and $\frac{N+1}{2}$ -th column.

A natural question arises: does the Cross Kernel have sufficient representational capacity to be used in practice? First, prior pruning studies [62], [63] have shown that corner weights are often pruned, suggesting they are not critical to model function. Second, we empirically evaluate the Cross Kernel by replacing all standard kernels with Cross Kernels, and observe at most a 0.32% minimal impact on accuracy—demonstrating that the Cross Kernel retains sufficient feature extraction capability for practical use.

3) **StriaBlock**: We now consolidate the proposed components. Table III compares the HE complexity of a standard convolutional layer with that of StriaNet convolutional layer, which incorporates proposed both the exRot-Free pattern and the Cross Kernel design. The StriaNet convolutional layer offers substantial advantages by completely eliminating the need for ex-Rot operations and significantly reducing the in-Rot operations. In addition to Rotation, other HE operations include Mult and Add. As shown in the table, the complexity of Mult inherently benefits from optimizations applied to Rotation. Moreover, the Add operation has a minimal impact on the overall cost—contributing as little as 0.5% [7], [8]—and is thus considered negligible in our analysis.

TABLE III
COMPLEXITY COMPARISON.

	#in-Rot	#ex-Rot (2 MIMO schemes)	#Mult
Vanilla Conv	$c_i(k_w k_h - 1)$ c_n	$c_o(c_n - 1)$ or $c_i(c_n - 1)k_w k_h$ c_n	$c_i c_o k_w k_h$ c_n
StriaNet Conv	$c_i(k_w + k_h - 1)$ c_n	0	$c_i c_o k_w k_h$ c_n^2

Next, we explore the detailed structure of the StriaBlock using a concise example, as illustrated in Figure 7. Conv0 is a convolutional layer comprising four input and four output channels, constructed with 3×3 kernels. It incorporates both the exRot-Free pattern and the Cross Kernel design, representing the *Inference Form* that is ultimately deployed on the server. In the figure, grey-highlighted cells denote active weights, while red-hatched cells indicate non-weight

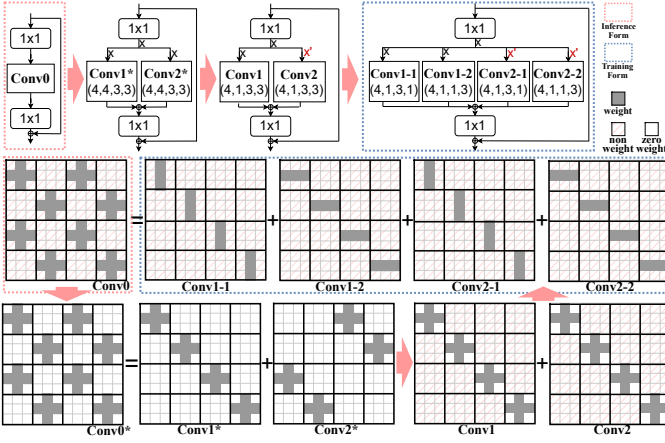


Fig. 7. StriaBlock Structural Details. Joint exRot-Free Pattern and Cross Kernel ($c_i = c_o = 4, k_w = k_h = 3, c_n = 2$). “=” indicates mathematical equivalence and “ $\Rightarrow$ ” denotes functional transformation equivalence.

positions—these elements are excluded from the model architecture and have no associated parameters. To deconstruct Conv0 and transition it from the *Inference Form* to a training-compatible form, we conceptually outline a three-step transformation process:

- **Conv0 $\Rightarrow$ Conv0* = Conv1* + Conv2*:** In the first step, all non-weight cells in Conv0 are replaced with zero-weight cells, yielding Conv0*. This introduces zero-valued weights in previously unused positions without affecting the computational behavior or functionality of the structure. Conv0* can then be decomposed into two components: Conv1* and Conv2*.
- **Conv1* + Conv2* $\Rightarrow$ Conv1 + Conv2:** In the second step, the input channel order for Conv2* is rearranged according to the manner as illustrated in Figure 4, transforming the input from x to x' . Additionally, all zero-weight kernels are replaced with non-weight kernels. This results in Conv1 and Conv2, which are two Depthwise Separable convolutional layers.
- **Conv1 + Conv2 $\Rightarrow$ Conv1-1 + Conv1-2 + Conv2-1 + Conv2-2:** In the final step, following the methodology shown in Figure 6, each Cross Kernel is decomposed into two sub-kernels. This transforms the combined structure (Conv1 + Conv2) into four distinct convolutional layers: Conv1-1, Conv1-2, Conv2-1, and Conv2-2. This configuration represents the *Training Form*, which is used during model training. Importantly, this form is fully equivalent to the original *Inference Form* (Conv0), ensuring a seamless and lossless transition between training and inference.

B. HE-Efficient Architectural Principle

In this section, we present the architectural principles of StriaNet. Unlike plaintext model architectures, which are not well-suited for HE, our design is specifically crafted to accommodate the unique computational characteristics of DL models in HE setting. We begin with a comprehensive analysis of the computation patterns in HE-based DL models to establish the fundamental relationship between HE operation complexity and key architectural factors. Subsequently, we evaluate full DL models within the HE context, with particular attention to

a critical variable: the ciphertext’s channel packing capacity, c_n , which varies with network depth.

1) **Network Architecture HE Complexity Analysis:** In plaintext scenarios, convolution is performed using a series of multiply-add operations. The computational complexity in the plaintext domain is quantified in terms of FLOPs, as illustrated in the following equations:

$$\begin{aligned} FLOPs &= (2c_i \times k_w k_h - 1) \times HW \times c_o, \\ &\approx 2k_w k_h \times c_i c_o \times HW, \end{aligned} \quad (1)$$

which exhibits one characteristic: *The plaintext cost of a convolutional layer is positively correlated with multiple factors*, including c_i (input channels), c_o (output channels), k_w (kernel width), k_h (kernel height), and HW (feature map size), with each factor contributing equally to the overall cost.

However, this rule becomes invalid in ciphertext scenarios. In plaintext scenarios, the complexity of a convolutional layer is positively correlated with all factors— c_i , c_o , k_w , k_h , HW —with the equal degree of influence. However, in ciphertext scenarios, we observe that **the complexity of HE operations is influenced exclusively by specific factors rather than all of them**. The following equations represent the complexity of in-Rot and ex-Rot, where $exRot_1$ and $exRot_2$ denote the complexity of ex-Rot in Grouped Out-Rotation MIMO and In-Rotation MIMO, and $exRot$ represents the integrated complexity:

$$inRot = \frac{c_i}{c_n} (k_w k_h - 1), \quad (2)$$

$$exRot_1 = \frac{c_o}{c_n} (c_n - 1), \quad (3)$$

$$exRot_2 = \frac{c_i}{c_n} (c_n - 1) k_w k_h, \quad (4)$$

$$exRot = \frac{c_n - 1}{c_n} \times \min\langle k_w k_h c_i, c_o \rangle, \quad (5)$$

First, as shown in Eq. 2, the complexity of in-Rot operations is solely determined by the number of input channels c_i and is independent of the number of output channels c_o . Next, the complexity of ex-Rot is given by Eq. 5, where $\min\langle a, b \rangle$ denotes the smaller of the two values, indicating that the MIMO scheme with the lower complexity is selected. Under this formulation, for layers with a kernel size of 1×1 , the ex-Rot complexity depends on a single dimension—either c_i or c_o —depending on which is smaller. In contrast, for layers with larger kernels, such as 3×3 or 5×5 , the ex-Rot complexity becomes correlated with c_o , as c_o is typically smaller than $9c_i$ or $25c_i$, respectively. Overall, for any given convolutional layer, the HE Rotation complexity is governed by only one dominant factor—either c_i or c_o . As a result, optimizing the remaining factors has negligible impact, rendering such efforts ineffective in the ciphertext setting. A representative counterexample is DenseNet. Its high-level design aims to shift computational cost from 3×3 layers to 1×1 layers, based on the assumption that the former incurs approximately $9 \times$ more FLOPs than the latter in plaintext settings. However, as previously discussed, HE rotation complexity is determined solely by either c_i or c_o , and is independent of the kernel size

$k_w \times k_h$. Consequently, this design choice offers no advantage under HE and even be detrimental, as it introduces additional HE cost in 1×1 layers. Therefore, our first architectural principle is:

• **Focused Constraint Principle:** To constrain the factors directly impacting the dominant computational cost, while allowing the remaining factors to vary freely, maximizing network performance capacity.

2) **Channel Packing Aware Network Scaling:** During the full network evaluation for HE efficiency, we observe several distinct differences compared to the plaintext setting. First, unlike in the plaintext setting where each operation incurs a uniform computational cost, i.e., *every multiply-add operation requires the same runtime* in the HE setting, **the cost of HE operations is not uniform**. As discussed earlier, in-Rot operations associated with corner weights are significantly more expensive than others. Moreover, the average costs of in-Rot and ex-Rot operations vary across the network depth. As shown in Table IV, in-Rot operations are generally more costly in shallower layers, whereas ex-Rot operations tend to dominate the computational cost in deeper layers of network.

TABLE IV

VARIATION IN AVERAGE EX-ROT, IN-ROT COST WITH MODEL DEPTH (SHALLOW LAYER $\rightarrow$ DEEP LAYERS: $c_n=2 \rightarrow c_n=512$).

# c_n	2	8	32	128	512
Avg in-Rot cost(ms)	16.10	13.13	11.48	9.68	8.82
Avg ex-Rot cost(ms)	5.71	7.50	11.30	14.50	18.93

We also observe that certain layers are highly sensitive to changes in dimensionality—small increases can lead to sharp surges in HE cost. Upon further investigation, we find that the fluctuation in HE complexity also varies with network depth. These insights motivate us to identify the key contributing factors and to develop corresponding architectural design principles.

Similar to other existing DL models constructed using fundamental components, StriaNet is architected with the proposed building block, StriaBlock. Figure 8 illustrates a StriaBlock on the left side. Its complexity of the HE rotation operations, which comprise both in-Rot and ex-Rot, can be expressed as:

$$Rot_{(StriaBlock)} = \left(\frac{4}{c_n} \times e \times D \right) + \left(\frac{c_n - 1}{c_n} \times 2 \times D \right), \quad (6)$$

where e is the **scaling factor**, which represents the extension ratio of the inverse-bottleneck structure. Next, we simplify Eq. 6 to highlight the relationship between given dimension D and the Rotation complexity of StriaBlock, resulting in:

$$Rot_{(StriaBlock)} = 2 \times \left(\frac{c_n - 1}{c_n} + \frac{2}{c_n} \times e \right) \times D, \quad (7)$$

The polynomial coefficient, $2 \times \left(\frac{c_n - 1}{c_n} + \frac{2}{c_n} \times e \right)$, represents the proportionality coefficient governing the relationship between dimension D and the complexity of Rotation operations. This relationship is graphically depicted on the right side of Figure 8. In shallow layers (e.g., when $c_n = 2$), even small changes in the scaling factor e result in substantial increases

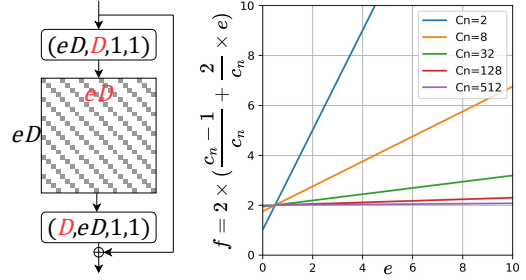


Fig. 8. Dimensions of StriaBlock.

in the proportionality coefficient. In contrast, in deeper layers (e.g., when $c_n = 512$), the proportionality coefficient remains relatively stable and exhibits low sensitivity to changes in e . These observations highlight that a key factor contributing to the fluctuation in HE cost across network depths is the ciphertext's channel packing capacity, c_n , which varies with depth. In DL models, down-sampling progressively reduces the size of input features. Under packed HE settings, this leads to an increase in c_n , meaning more channels are packed into a single ciphertext. This, in turn, influences the complexity of HE operations. This insight motivates the formulation of our second architectural principle:

• **Channel Packing-Aware Scaling Principle:** Dynamically adjust the scaling factor e , the bottleneck ratio of each building block, based on the ciphertext's channel packing capacity, c_n . Specifically, e should be constrained in shallow layers to mitigate cost spikes and progressively increased in deeper layers to fully exploit the available network capacity.

Following this architectural principle, we construct the network using the proposed building blocks. Table V outlines our architecture for the ImageNet dataset, featuring a scaling factor e that varies from 2 to 8 in alignment with network depth. Detailed performance evaluations are presented in the evaluation section.

TABLE V

STRIANET ARCHITECTURES FOR IMAGENET.

Layers	Output	Cn	StriaNet-L1	StriaNet-L2	StriaNet-L3	StriaNet-L4
Conv	112×112		7×7, 32, stride 2			
Pooling	56×56	2	3×3, Max pool, stride 2			
StriaBlock (1:2)	56×56	2	$\begin{pmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 32 \end{pmatrix} \times 2$	$\begin{pmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 32 \end{pmatrix} \times 3$	$\begin{pmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 32 \end{pmatrix} \times 3$	$\begin{pmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 32 \end{pmatrix} \times 3$
StriaBlock (1:4)	28×28	8	$\begin{pmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 64 \end{pmatrix} \times 2$	$\begin{pmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 64 \end{pmatrix} \times 4$	$\begin{pmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 64 \end{pmatrix} \times 4$	$\begin{pmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 64 \end{pmatrix} \times 8$
StriaBlock (1:6)	14×14	32	$\begin{pmatrix} 1 \times 1, 768 \\ 3 \times 3, 768 \\ 1 \times 1, 128 \end{pmatrix} \times 4$	$\begin{pmatrix} 1 \times 1, 768 \\ 3 \times 3, 768 \\ 1 \times 1, 128 \end{pmatrix} \times 6$	$\begin{pmatrix} 1 \times 1, 768 \\ 3 \times 3, 768 \\ 1 \times 1, 128 \end{pmatrix} \times 12$	$\begin{pmatrix} 1 \times 1, 768 \\ 3 \times 3, 768 \\ 1 \times 1, 128 \end{pmatrix} \times 33$
StriaBlock (1:8)	7×7	128	$\begin{pmatrix} 1 \times 1, 2048 \\ 3 \times 3, 2048 \\ 1 \times 1, 256 \end{pmatrix} \times 2$	$\begin{pmatrix} 1 \times 1, 2048 \\ 3 \times 3, 2048 \\ 1 \times 1, 256 \end{pmatrix} \times 3$	$\begin{pmatrix} 1 \times 1, 2048 \\ 3 \times 3, 2048 \\ 1 \times 1, 256 \end{pmatrix} \times 3$	$\begin{pmatrix} 1 \times 1, 2048 \\ 3 \times 3, 2048 \\ 1 \times 1, 256 \end{pmatrix} \times 3$
Conv	7×7	128	1×1, 2048			
FC	1×1		Avg pool 1000D, FC, softmax			

C. Comparing with Depthwise Convolution

The building blocks of Xception [33] and MobileNet [34], when utilizing the simplest form of depthwise convolution, incidentally achieve an exRot-free configuration, corresponding to **exRot-free pattern with extension = 1**. However, our **exRot-free pattern typically employs larger extensions**—e.g., **extension = 4**—as illustrated in Figure 9. This pattern

Layers	Output	Cn	StriaNet-M1	StriaNet-M2
StriaBlock	64×64	2	$\begin{pmatrix} 1 \times 1, 128 \\ 3 \times 3, 64 \\ 1 \times 1, 32 \end{pmatrix} \times 3$	$\begin{pmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 32 \end{pmatrix} \times 3$
StriaBlock	32×32	8	$\begin{pmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 64 \end{pmatrix} \times 4$	$\begin{pmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 64 \end{pmatrix} \times 4$
StriaBlock	16×16	32	$\begin{pmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \end{pmatrix} \times 6$	$\begin{pmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \end{pmatrix} \times 6$

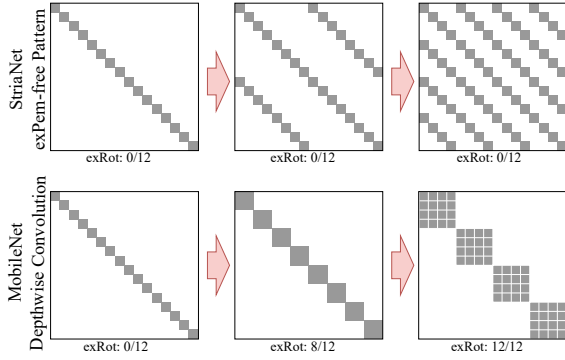


Fig. 9. ExRot-Free Pattern and Depthwise Convolution.

introduces additional stria patterns while consistently maintaining the exRot-free configuration. Conversely, depthwise convolution disrupts the exRot-free configuration by enhancing the main kernel diagonal, increasing kernel group sizes from 1×1 to 4×4 . This disruption significantly raises the number of ex-Rot operations—from 0 ex-Rot to 8 ex-Rot, and eventually to 12 ex-Rot. In practical implementations, the extension can go as high as 32, where an extended MobileNet would require 224 ex-Rot. This distinction arises from the fundamentally different designs of our StriaBlock and depthwise convolution. The StriaBlock featuring the exRot-Free pattern, is specifically designed for MIMO schemes in HE setting, enabling us to circumvent the resource-intensive ex-Rot operation. In contrast, depthwise convolution is primarily designed for lightweight filtering in plaintext setting, employing a single convolutional kernel per input channel to reduce the computation of feature extraction.

IV. PERFORMANCE EVALUATION

All models are trained using PyTorch [64] without any additional enhancement techniques, such as advanced training strategies or data preprocessing methods (e.g., cleaning or augmentation), on servers with NVIDIA A100 GPUs. Detailed training configurations are provided in Appendix-D. The trained StriaNet is deployed using the SEAL library [60], [65] and the Secure and Correct Inference (SCI) library from EzPC [66], evaluated on a testbed featuring an AMD 3995 CPU². All performance evaluations are conducted with parameters set to match the default configurations used in existing state-of-the-art frameworks [7], [8], ensuring a fair comparison. A comprehensive evaluation is carried out on three widely used datasets of varying scales: large (ImageNet [67]), medium (Tiny ImageNet [68]), and small (CIFAR-10 [69]). The analysis primarily emphasizes linear computational costs in convolutional layers, as they constitute the dominant component of end-to-end secure CNNs inference, with Rotation and Mult operations being the primary contributors.

A. ImageNet Classification

We evaluate the StriaNet-Large (L) Series, specifically L1-L4, on the ImageNet dataset, comparing it with classic backbone neural networks including VGG [29], ResNet [21],

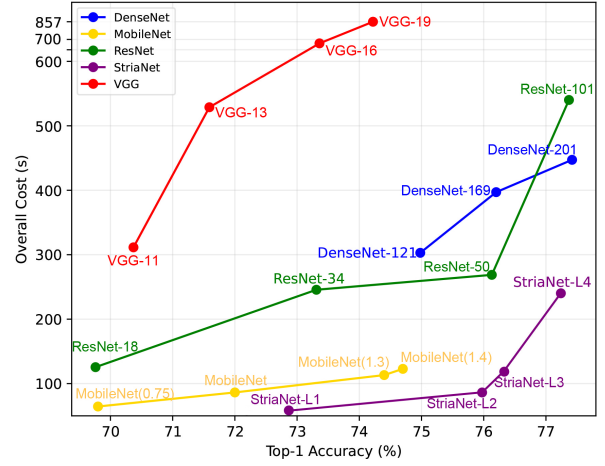


Fig. 10. Performance curve of StriaNet-L Series compared to Series of VGG, ResNet, DenseNet and MobileNet. Overall costs are under LAN conditions.

DenseNet [30], and MobileNet v2 (with varying width multipliers) [31], [70]. Figure 10 offers a visual comparison, while Table VI summarizes the detailed performance metrics. At comparable accuracy levels, StriaNet achieves up to a $9.78\times$ speedup over VGG, and $3.35\times$ and $2.25\times$ speedups over DenseNet and ResNet, respectively. Compared to the lightweight MobileNet, StriaNet demonstrates a significant performance advantage under the same cost. For instance, StriaNet-L2 outperforms MobileNet by 4% in accuracy under equivalent resource constraints.

In addition to overall performance, our architecture, designed with a strong focus on Rotation optimizations, achieves significantly lower Rotation complexity compared to networks originally designed for plaintext scenarios. As shown in Table VI, several groups of Rotation counts are highlighted in different colors to illustrate these comparisons.

Table VI also presents two specific performance metrics: the HE cost per unit parameter (Cost/Params) and the HE cost per unit performance (Cost/Acc). These metrics provide deeper insights into each models' efficiency and cost effectiveness under HE scenarios. For fairness, comparisons are also conducted at matched accuracy levels, indicated by different color annotations. For the **Cost/Params** metric, VGG unsurprisingly offers no advantage. DenseNet and MobileNet exhibit nearly 200% the inefficiency compared to StriaNet in this regard. Notably, ResNet achieves a similar level of per-parameter efficiency to StriaNet. However, due to its significantly larger parameter count, approximately $2.8\times$ more, it incurs a substantially higher overall HE cost. It is also noteworthy that although MobileNet exhibits the lowest total HE cost among the four plaintext networks, its relatively high Cost/Params value suggests limited scalability. This implies that as the model size and parameter count grow, the HE cost of MobileNet will rise rapidly, potentially surpassing even that of ResNet and further widening the efficiency gap with StriaNet. For the **Cost/Acc** metric, StriaNet demonstrates a clear comparative advantage over all competing plaintext networks. Its HE cost per unit of performance can be as low as 29.75% of the baseline, underscoring the exceptional

²Unless otherwise specified, 4 out of 128 CPU threads are utilized.

TABLE VI
PERFORMANCE RESULTS COMPARISON ON IMAGENET CLASSIFICATION.

Models	Top-1 Acc	Params	#in-Rot	#ex-Rot	#Mult	Commu. Size	Overall Runtime (LAN, WAN)		Cost /Params	Cost /Acc
VGG-19	74.22%	20.02M	11,968	4,704	2,668K	858.7MB	857.35s	905.30s	43	1155
VGG-16	73.36%	14.71M	11,584	3,216	2,447K	834.2MB	693.08s	739.79s	47	945
VGG-13	71.59%	9.4M	9,920	2,144	1,783K	728.0MB	528.81s	570.17s	56	739
VGG-11	70.37%	9.22M	3,776	2,144	1,193K	335.9MB	311.43s	333.05s	34	443
ResNet-101	77.37%	39.63M	2,848	46,548	1,186K	1107.8MB	540.09s	596.33s	14	698
ResNet-50	76.13%	20.69M	1,760	21,252	594K	722.9MB	268.76s	305.06s	13	353
ResNet-34	73.31%	21.1M	3,408	7,112	581K	219.4MB	245.61s	256.75s	12	335
ResNet-18	69.76%	10.99M	1,808	3,600	286K	117.3MB	125.53s	131.53s	11	180
DenseNet-201	77.42%	18.72M	6,400	18,636	648K	1461.8MB	447.22s	505.45s	24	578
DenseNet-169	76.20%	16.06M	5,888	16,156	565K	1236.8MB	397.07s	447.19s	25	521
DenseNet-121	74.98%	9.3M	5,504	10,852	478K	1034.1MB	302.92s	345.74s	33	404
MobileNet(1.4)	74.70%	5.07M	4,248	4,546	108K	655.8MB	122.88s	157.91s	24	164
MobileNet(1.3)	74.40%	4.49M	3,912	4,344	92K	604.3MB	113.23s	145.53s	25	152
MobileNet(1.0)	72.00%	2.98M	2,925	3,556	54K	448.4MB	86.31s	110.06s	29	120
MobileNet(.75)	69.80%	1.55M	2,628	2,498	38K	402.2MB	64.48s	85.84s	42	92
StriaNet-L4	77.24%	14M	4,768	10,716	439K	1384.8MB	240.14s	311.16s	17	311
StriaNet-L3	76.33%	7.65M	2,240	5,060	213K	677.2MB	118.53s	154.06s	15	155
StriaNet-L2	76.02%	5.92M	1,664	3,572	159K	517.9MB	87.68s	115.19s	15	115
StriaNet-L1	72.79%	3.44M	1,024	2,312	106K	333.1MB	58.64s	76.84s	15	80

*The network bandwidth: LAN 3Gbps WAN: 400Mbps.

efficiency of networks specifically designed for HE settings compared to plaintext-originated architectures.

TABLE VII
BUILDING BLOCK COMPLEXITY COMPARISON

Depth	Input	Metric	ResNet	MobileNet	StriaNet
Deep Blocks ← Shallow Blocks	56×56	#in-Rot	256	576	128
		#ex-Rot	96	24	32
		# Rotation	352	600	160
	28×28	#in-Rot	128	192	128
		#ex-Rot	336	56	112
		# Rotation	464	248	240
	14×14	#in-Rot	64	144	96
		#ex-Rot	744	186	248
		# Rotation	808	330	344
	7×7	#in-Rot	32	64	64
		#ex-Rot	1524	508	508
		# Rotation	1556	572	572

Building Block Complexity Benchmark. As ResNet, MobileNet, and StriaNet all adopt building blocks as their fundamental architectural units, we conduct a benchmark evaluation of their respective blocks. The results, presented in Table VII, compare their HE Rotation complexity across different depths, ranging from shallow network with large input (56×56) to deep network with smaller input sizes (7×7). For both ResNet and StriaNet, the Rotation complexity of their building blocks increases with network depth. However, due to optimized design such as the exRot-Free pattern, StriaNet consistently exhibits lower Rotation complexity. In contrast, MobileNet’s building blocks show noticeable spikes of Rotation complexity in shallow layers, primarily due to the high number of in-Rot. StriaNet, on the other hand, maintains stable Rotation complexity throughout the network. This stability is attributed to its dynamic scaling factor, which adjusts the bottleneck ratio of each building block with network depths, combined with in-Rot optimization provided by the Cross Kernel design.

TABLE VIII
ABLATION STUDY

	CPAS	ExRot-free	Cross Kernel	times(s)
StriaNet-L3	-	-	-	1797.91
	✓	-	-	971.20 (46%↓)
	✓	✓	-	169.37 (83%↓)
	✓	✓	✓	118.53 (30%↓)

*The “↓” denotes the relative reduction compared to the row above.

Ablation Study. Although architecture functions as a unified system with jointly contributing components, the ablation study highlights the individual contributions of each design

element. As shown in Table VIII, the ExRot-free Pattern contributes significantly to improvements. Moreover, Channel Packing-Aware Scaling (CPAS) enhances the network by optimizing dimensional configuration, resulting in twofold enhancement. Lastly, Cross Kernel further boosts model efficiency by specifically targeting in-Rot optimization.

TABLE IX
PEAK MEMORY USAGE FOR CIPHERTEXT STORAGE

Model	HW, c_o, c_i, k, s	Memory Overhead
ResNet	$56^2, 64, 64, 3, 1$	324MB
DenseNet	$56^2, 32, 128, 3, 1$	648MB
MobileNet	$56^2, 144, 144, 3, 1$	729MB
StriaNet	$56^2, 64, 64, 3, 1$	180MB

Memory Overhead. Memory overhead in secure inference is measured by peak memory usage, related to the maximum number of ciphertexts stored simultaneously. Table IX presents memory overhead for the implementations of ResNet, MobileNet, DenseNet, and StriaNet with the ImageNet dataset. The second column details where the peak memory occurs, along with the corresponding configurations. Compared to other architectures, StriaNet requires only 180MB of memory, demonstrating a significantly reduced memory footprint.

TABLE X
RESULTS ON TINY IMAGENET, CIFAR-10 CLASSIFICATION.

	Models	Top-1 Acc	Rotation (s)	Mult (s)	Overall (s)
Tiny ImageNet	ResNet-50	65.50%	150.69	140.00	254.69
	<i>ResNet-50-Pruned</i>	65.54%	70.29	36.13	106.43
	VGG-16	65.00%	77.89	254.31	323.20
	<i>VGG-16-Pruned</i>	64.94%	41.36	56.01	97.37
	MobileNetV2	67.37%	107.55	21.91	129.46
	StriaNet-M2	68.57%	51.91	31.67	83.58
CIFAR-10	StriaNet-M1	67.49%	34.63	19.13	53.76
	ResNet-50	93.50%	172.35	27.79	200.14
	VGG-16	94.50%	58.37	61.34	119.72
	<i>VGG-16-Pruned</i>	94.41%	25.29	16.40	41.69
	ResNet-56-CIFAR	93.03%	47.38	13.57	60.95
	StriaNet-S2	95.14%	24.96	7.82	32.78
	StriaNet-S1	94.56%	13.43	8.24	21.67

B. Tiny ImageNet and CIFAR-10 Classification

We also evaluate StriaNet’s performance on the Tiny ImageNet and CIFAR-10 datasets, as presented in Table X. On both benchmarks, StriaNet demonstrates exceptional cost efficiency, achieving speedups of $6.01 \times$ and $9.24 \times$, respectively. Notably, compared to pruned ResNet-50 [27], StriaNet is **200%** faster

with improving accuracy by 2%. This underscores the effectiveness of architectures tailored for HE settings over those adapted from plaintext models, establishing StriaNet as a new benchmark for cost-efficient HE inference. For architectural details of the StriaNet-M and StriaNet-S series and more performance details, please refer to Appendix-C.

C. Exploration of StriaNet with Cheetah

An important design focus of our work is minimizing HE Rotations. Cheetah [5] adopts a specialized encoding and polynomial multiplication scheme to perform convolutions directly, thereby implicitly achieving element alignment without explicit Rotation. However, this prevents efficient arrangement and alignment of elements within the output ciphertext, resulting in low ciphertext utilization and substantial communication overhead [9], [10], [71], [72].

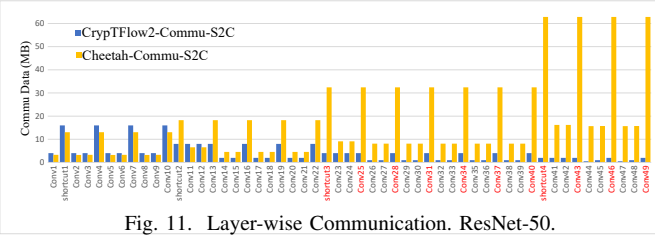


Fig. 11. Layer-wise Communication. ResNet-50.

Figure 11 presents the layer-wise communication data transmitted from the server to the client. It is evident that, as the model depth increases, Cheetah incurs significantly higher communication overhead compared to CrypTFlow2—reaching up to $31.4\times$ more for certain layers (highlighted in red). This overhead can cause substantial delays in real-world applications, particularly in scenarios with limited network bandwidth, such as WAN or wireless networks. However, replacing Cheetah with original CrypTFlow2 introduces increased computational costs due to the heavy use of Rotation operations, as shown in Table XI. Our Rotation-Free Pattern provides a promising solution to this trade-off, offering an architecture that minimizes HE Rotations without incurring additional communication overhead.

TABLE XI
MICROBENCHMARKS.

Method	HW, c_o, c_t, k, s	Comp. Time (Sec)	S2C Commu. (MB)
Cheetah	$28^2, 1024, 512, 1, 2$	2.92	32.4
	$14^2, 1024, 256, 1, 1$	1.58	32.4
	$14^2, 2048, 1024, 1, 2$	3.14	62.8
	$7^2, 2048, 512, 1, 1$	1.86	62.8
CrypTFlow2	$28^2, 1024, 512, 1, 2$	6.25	4.0
	$14^2, 1024, 256, 1, 1$	3.45	4.0
	$14^2, 2048, 1024, 1, 2$	7.34	2.0
	$7^2, 2048, 512, 1, 1$	4.54	2.0
Ours	$28^2, 1024, 512, 1, 2$	3.12	4.0
	$14^2, 1024, 256, 1, 1$	1.51	4.0
	$14^2, 2048, 1024, 1, 2$	3.21	2.0
	$7^2, 2048, 512, 1, 1$	1.77	2.0

The end-to-end evaluations are summarized in Table XII, where we replace Cheetah with our Rotation-Free Pattern for layers exhibiting high communication overhead, evaluated under varying computational capacities and real-world network conditions [73]. The results show that our design effectively alleviates Cheetah’s communication bottleneck in practical

TABLE XII
PERFORMANCE COMPARISON.

Model Data	Server CPU Thrs	Comp & Commu Runtime (Sec): Ours/Vanilla Cheetah			
		WAN	5G	4G/LTE	IoT
RsNt50 cfr10	4×	72.67/81.46 (89.21%)	83.29/100.46 (82.91%)	104.53/138.47 (75.49%)	231.97/366.49 (63.30%)
	8×	42.12/50.71 (83.06%)	52.74/69.71 (75.66%)	73.98/107.71 (68.68%)	201.42/335.74 (59.99%)
	16×	26.61/35.10 (75.82%)	37.23/54.10 (68.82%)	58.47/92.11 (63.48%)	185.91/320.13 (58.07%)
RsNt50 tiny IN	4×	76.46/85.18 (89.76%)	88.91/106.01 (83.87%)	113.81/147.68 (77.07%)	263.21/397.66 (66.19%)
	8×	44.94/53.50 (84.01%)	57.39/74.33 (77.21%)	82.29/115.99 (70.95%)	231.69/365.98 (63.31%)
	16×	28.95/37.42 (77.36%)	41.40/58.25 (71.07%)	66.30/99.91 (66.35%)	215.70/349.90 (61.65%)
RsNt101 tiny IN	4×	143.97/162.74 (88.47%)	165.69/201.76 (82.12%)	209.15/279.80 (74.75%)	469.86/748.07 (62.81%)
	8×	83.78/101.82 (82.28%)	105.50/140.84 (74.91%)	148.96/218.89 (68.05%)	409.67/687.16 (59.62%)
	16×	53.23/70.91 (75.07%)	74.96/109.93 (68.19%)	118.41/187.98 (62.99%)	379.12/656.25 (57.77%)

*The CPU contains a total of $128\times$ threads. The speed of each network is: WAN: 400Mbps, 5G: 200Mbps, 4G/LTE: 100Mbps, IoT: 25Mbps [73].

deployment scenarios, achieving an overall $1.73\times$ speedup in end-to-end inference.

Note that work [11] mitigates Cheetah’s communication issue through an additional post-processing “interleave” step, which fundamentally involves rotations over polynomial coefficients [74]. In contrast, our approach fully leverages the optimization potential of standard HE protocol by employing the Rotation-Free Pattern to achieve Rotation optimization inherently. This results in a more concise and direct solution. Furthermore, our design does not rely on any specialized protocol, offering improved compatibility and generality.

V. CONCLUSION AND FUTURE WORK

This work addresses a critical gap in current privacy-preserving MLaaS research—the lack of neural network architectures explicitly designed for HE settings. We argue that significant efficiency gains can be achieved by designing networks specifically tailored to the unique computational constraints of HE, rather than adapting existing plaintext models. We begin by introducing StriaBlock, a novel building block targeting the most computationally expensive HE operation—rotation. StriaBlock integrates ExRot-Free Convolution with a novel Cross Kernel, entirely eliminating the need for external Rotations and reducing internal Rotation usage to just 19% of that required by plaintext models. Next, we propose the architectural principle, includes the Focused Constraint Principle, which limits cost-sensitive factors while preserving flexibility in others, and the Channel Packing-Aware Scaling Principle, which dynamically adapts bottleneck ratios based on ciphertext channel capacity that varies with network depth. These strategies efficiently control the local and overall HE cost, enabling a balanced architecture for HE settings. The resulting network, StriaNet, is comprehensively evaluated and achieves comparable accuracy while delivering speedups of $9.78\times$, $6.01\times$, and $9.24\times$ on ImageNet, Tiny ImageNet, and CIFAR-10, respectively. For future work, we aim to further enhance StriaNet by exploring the incorporation of Neural Architecture Search techniques.

ACKNOWLEDGMENT

The authors would like to express thanks of gratitude to the lab members for helping and collaborating on this work and anonymous reviewers for the constructive comments. The work of H. Wu was supported in part by the National Science Foundation under Grants SaTC-2439013, CNS-2413009, DGE-2336109, OAC-2320999, IIS-2236578, and CNS-2120279.

REFERENCES

- [1] W. Wang, S. Wang, J. Gao, M. Zhang, G. Chen, T. K. Ng, and B. C. Ooi, "Rafiki: Machine learning as an analytics service system," *arXiv preprint arXiv:1804.06087*, 2018.
- [2] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing, "Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy," in *Proceedings of the ICML*, 2016, pp. 201–210.
- [3] P. Mohassel and Y. Zhang, "Secureml: A system for scalable privacy-preserving machine learning," in *Proceedings of the IEEE S&P*, 2017, pp. 19–38.
- [4] J. Liu, M. Juuti, Y. Lu, and N. Asokan, "Oblivious neural network predictions via minionn transformations," in *Proceedings of the ACM SIGSAC*, 2017, pp. 619–631.
- [5] C. Juvekar, V. Vaikuntanathan, and A. Chandrakasan, "{GAZELLE}: A low latency framework for secure neural network inference," in *Proceedings of the USENIX Security*, 2018, pp. 1651–1669.
- [6] P. Mishra, R. Lehmkuhl, A. Srinivasan, W. Zheng, and R. A. Popa, "Delphi: A cryptographic inference service for neural networks," in *Proceedings of the USENIX Security*, 2020, pp. 2505–2522.
- [7] D. Rathee, M. Rathee, N. Kumar, N. Chandran, D. Gupta, A. Rastogi, and R. Sharma, "Cryptflow2: Practical 2-party secure inference," in *Proceedings of the ACM SIGSAC*, 2020, pp. 325–342.
- [8] Z. Huang, W. jie Lu, C. Hong, and J. Ding, "Cheetah: Lean and fast secure Two-Party deep neural network inference," in *31st USENIX Security Symposium (USENIX Security 22)*. Boston, MA: USENIX Association, Aug. 2022, pp. 809–826. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/huang-zhichong>
- [9] M. Hao, H. Li, H. Chen, P. Xing, G. Xu, and T. Zhang, "Iron: Private inference on transformers," *Advances in Neural Information Processing Systems*, vol. 35, pp. 15 718–15 731, 2022.
- [10] Q. Pang, J. Zhu, H. Möllering, W. Zheng, and T. Schneider, "Bolt: Privacy-preserving, accurate and efficient inference for transformers," *Cryptology ePrint Archive*, 2023.
- [11] W.-j. Lu, Z. Huang, Z. Gu, J. Li, J. Liu, C. Hong, K. Ren, T. Wei, and W. Chen, "Bumblebee: Secure two-party inference framework for large transformers," *Cryptology ePrint Archive*, 2023.
- [12] J. Fan and F. Vercauteren, "Somewhat practical fully homomorphic encryption," *IACR Cryptol. ePrint Arch.*, p. 144, 2012.
- [13] M. Bellare, V. T. Hoang, and P. Rogaway, "Foundations of garbled circuits," in *Proceedings of the ACM CCS*, 2012, pp. 784–796.
- [14] G. Brassard, C. Crépeau, and J.-M. Robert, "All-or-nothing disclosure of secrets," in *Proceedings of the EUROCRYPT*, 1986, pp. 234–238.
- [15] A. Shamir, "How to share a secret," *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.
- [16] C. Gentry, "Fully homomorphic encryption using ideal lattices," in *Proceedings of the ACM STOC*, 2009, pp. 169–178.
- [17] Z. Brakerski and V. Vaikuntanathan, "Efficient fully homomorphic encryption from (standard) lwe," *SIAM Journal on computing*, vol. 43, no. 2, pp. 831–871, 2014.
- [18] W. Ao and V. N. Boddeti, "{AutoFHE}: Automated adaption of {CNNs} for efficient evaluation over {FHE}," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 2173–2190.
- [19] D. Kim, J. Park, J. Kim, S. Kim, and J. H. Ahn, "Hyphen: A hybrid packing method and its optimizations for homomorphic encryption-based neural networks," *IEEE Access*, vol. 12, pp. 3024–3038, 2023.
- [20] Amazon.com, "Developer reference for alexa interface," 2021. [Online]. Available: <https://developer.amazon.com/en-US/docs/alexa/device-apis/alexa-interface.html>
- [21] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE CVPR*, 2016, pp. 770–778.
- [22] Q. Zhang, C. Xin, and H. Wu, "Gala: Greedy computation for linear algebra in privacy-preserved neural networks," *arXiv preprint arXiv:2105.01827*, 2021.
- [23] R. Ran, X. Luo, W. Wang, T. Liu, G. Quan, X. Xu, C. Ding, and W. Wen, "Spencnn: orchestrating encoding and sparsity for fast homomorphically encrypted neural network inference," in *International Conference on Machine Learning*. PMLR, 2023, pp. 28 718–28 728.
- [24] J. H. Cheon, M. Kang, T. Kim, J. Jung, and Y. Yeo, "Batch inference on deep convolutional neural networks with fully homomorphic encryption using channel-by-channel convolutions," *IEEE Transactions on Dependable and Secure Computing*, 2024.
- [25] E. Aharoni, A. Adir, M. Baruch, N. Drucker, G. Ezov, A. Farkash, L. Greenberg, R. Masalha, G. Moshkovich, D. Murik *et al.*, "Helayers: A tile tensors framework for large neural networks on encrypted data," *arXiv preprint arXiv:2011.01805*, 2020.
- [26] L. Folkerts, C. Gouert, and N. G. Tsoutsos, "Redsec: Running encrypted discretized neural networks in seconds," *Cryptology ePrint Archive*, 2021.
- [27] Y. Cai, Q. Zhang, R. Ning, C. Xin, and H. Wu, "Mosaic: A prune-and-assemble approach for efficient model pruning in privacy-preserving deep learning," in *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*, 2024, pp. 1034–1048.
- [28] W. Legiest, F. Turan, M. Van Beirendonck, J.-P. D'Anvers, and I. Verbauwhede, "Neural network quantisation for faster homomorphic encryption," in *2023 IEEE 29th International Symposium on On-Line Testing and Robust System Design (IOLTS)*. IEEE, 2023, pp. 1–3.
- [29] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2014.
- [30] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 4700–4708.
- [31] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 4510–4520.
- [32] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention—MICCAI 2015: 18th International Conference, Munich, Germany, October 5-9, 2015, Proceedings, Part III 18*. Springer, 2015, pp. 234–241.
- [33] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," 2017.
- [34] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.
- [35] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, "Squeezenet: Alexnet-level accuracy with 50x fewer parameters and 0.5 mb model size," *arXiv preprint arXiv:1602.07360*, 2016.
- [36] X. Zhang, X. Zhou, M. Lin, and J. Sun, "Shufflenet: An extremely efficient convolutional neural network for mobile devices," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 6848–6856.
- [37] D. Demmler, T. Schneider, and M. Zohner, "Aby-a framework for efficient mixed-protocol secure two-party computation," in *Proceedings of the NDSS*, 2015.
- [38] M. S. Riazi, C. Weinert, O. Tkachenko, E. M. Songhori, T. Schneider, and F. Koushanfar, "Chameleon: A hybrid secure computation framework for machine learning applications," in *Proceedings of the ASIACCS*, 2018, pp. 707–721.
- [39] B. D. Rouhani, M. S. Riazi, and F. Koushanfar, "Deepsecure: Scalable provably-secure deep learning," in *Proceedings of the DAC*, 2018, pp. 1–6.
- [40] P. Mohassel and P. Rindal, "Aby3: A mixed protocol framework for machine learning," in *Proceedings of the ACM SIGSAC*, 2018, pp. 35–52.
- [41] S. Wagh, S. Tople, F. Benhamouda, E. Kushilevitz, P. Mittal, and T. Rabin, "Falcon: Honest-majority maliciously secure framework for private deep learning," *arXiv preprint arXiv:2004.02229*, 2020.
- [42] A. Patra and A. Suresh, "Blaze: blazing fast privacy-preserving machine learning," *arXiv preprint arXiv:2005.09042*, 2020.

- [43] X. Jiang, M. Kim, K. Lauter, and Y. Song, "Secure outsourced matrix computation and application to neural networks," in *Proceedings of the ACM SIGSAC*, 2018, pp. 1209–1222.
- [44] M. S. Riaz, M. Samragh, H. Chen, K. Laine, K. E. Lauter, and F. Koushanfar, "Xonn: Xnor-based oblivious deep neural network inference," in *USENIX Security Symposium*, 2019, pp. 1501–1518.
- [45] S. Tan, B. Knott, Y. Tian, and D. J. Wu, "Cryptgpu: Fast privacy-preserving machine learning on the gpu," *arXiv preprint arXiv:2104.10949*, 2021.
- [46] S. Wagh, D. Gupta, and N. Chandran, "Securenn: 3-party secure computation for neural network training," *Proc. Priv. Enhancing Technol.*, vol. 2019, no. 3, pp. 26–49, 2019.
- [47] N. Kumar, M. Rathee, N. Chandran, D. Gupta, A. Rastogi, and R. Sharma, "Cryptflow: Secure tensorflow inference," in *Proceedings of the IEEE S&P*, 2020, pp. 336–353.
- [48] Q. Zhang, C. Wang, H. Wu, C. Xin, and T. V. Phuong, "Gelu-net: A globally encrypted, locally unencrypted deep neural network for privacy-preserving learning," in *Proceedings of the IJCAI*, 2018, pp. 3933–3939.
- [49] L. K. Ng and S. S. Chow, "Gforce: Gpu-friendly oblivious and rapid neural network inference," in *Proceedings of the USENIX Security*, 2021.
- [50] F. Boemer, R. Cammarota, D. Demmler, T. Schneider, and H. Yalame, "Mp2ml: a mixed-protocol machine learning framework for private inference," in *Proceedings of the ARES*, 2020, pp. 1–10.
- [51] A. Patra, T. Schneider, A. Suresh, and H. Yalame, "Aby2. 0: Improved mixed-protocol secure two-party computation," in *Proceedings of the USENIX Security*, 2021.
- [52] C. Dong, J. Wong, Y. Tong, J.-N. Liu, A. Yang, Y. Cheng, and S. Hu, "Fusion: Efficient and secure inference resilient to malicious server and curious clients," *arXiv preprint arXiv:2205.03040*, 2022.
- [53] F. Schroff, D. Kalenichenko, and J. Philbin, "Facenet: A unified embedding for face recognition and clustering," in *Proceedings of the IEEE CVPR*, 2015, pp. 815–823.
- [54] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Proceedings of the NeurIPS*, vol. 25, pp. 1097–1105, 2012.
- [55] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Proceedings of the ASIACRYPT*, 2017, pp. 409–437.
- [56] P. Paillier, "Public-key cryptosystems based on composite degree residuosity classes," in *Proceedings of the EUROCRYPT*, 1999, pp. 223–238.
- [57] Z. Brakerski, C. Gentry, and S. Halevi, "Packed ciphertexts in lwe-based homomorphic encryption," in *Proceedings of the PKC*, 2013, pp. 1–13.
- [58] M. Albrecht, M. Chase, H. Chen, J. Ding, S. Goldwasser, S. Gorbunov, S. Halevi, J. Hoffstein, K. Laine, K. Lauter *et al.*, "Homomorphic encryption standard," *Protecting privacy through homomorphic encryption*, pp. 31–62, 2021.
- [59] D. Natarajan and W. Dai, "Seal-embedded: A homomorphic encryption library for the internet of things," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 756–779, 2021.
- [60] "Microsoft SEAL (release 3.3)," <https://github.com/Microsoft/SEAL>, Jun. 2019, Microsoft Research, Redmond, WA.
- [61] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 2818–2826.
- [62] Y. Cai, Q. Zhang, R. Ning, C. Xin, and H. Wu, "Hunter: He-friendly structured pruning for efficient privacy-preserving deep learning," in *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*, 2022, pp. 931–945.
- [63] T. Chen, B. Ji, T. Ding, B. Fang, G. Wang, Z. Zhu, L. Liang, Y. Shi, S. Yi, and X. Tu, "Only train once: A one-shot neural network training and pruning framework," *Advances in Neural Information Processing Systems*, vol. 34, pp. 19637–19651, 2021.
- [64] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, and e. a. Chanan, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems* 32. Curran Associates, Inc., 2019, pp. 8024–8035. [Online]. Available: <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>
- [65] Hulse, "Microsoft seal 4.x for python," 2023. [Online]. Available: <https://github.com/Hulse/SEAL-Python>
- [66] M. Research, "Microsoft ezpc (easy secure multi-party computation)," 2019. [Online]. Available: <https://www.microsoft.com/en-us/research/project/ezpc-easy-secure-multi-party-computation/>
- [67] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei, "ImageNet Large Scale Visual Recognition Challenge," *IJCV*, vol. 115, no. 3, pp. 211–252, 2015.
- [68] Stanford.edu, "Tiny imagenet dataset," 2015. [Online]. Available: <http://cs231n.stanford.edu>
- [69] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.
- [70] X. D. Y. L. A. R. L. H. P. J. F. Y. F. L. J. K. Hongkun Yu, Chen Chen and J. Li, "TensorFlow Model Garden," 2020. [Online]. Available: github.com/tensorflow/models/tree/master/research/slim/nets/mobilenet
- [71] K. Gupta, D. Kumaraswamy, N. Chandran, and D. Gupta, "Llama: A low latency math library for secure inference," *Cryptology ePrint Archive*, 2022.
- [72] T. Xu, M. Li, R. Wang, and R. Huang, "Falcon: Accelerating homomorphically encrypted convolutions for efficient private mobile network inference," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2023, pp. 1–9.
- [73] SPEEDTEST.net, "Market analysis of mobile and fixed broadband speeds, us," 2024. [Online]. Available: <https://www.speedtest.net/global-index/united-states>
- [74] X. Hou, J. Liu, J. Li, Y. Li, W.-j. Lu, C. Hong, and K. Ren, "Ciphertgpt: Secure two-party gpt inference," *Cryptology ePrint Archive*, 2023.
- [75] pytorch.org, "Pytorch tutorials: Training with distributed data parallelism," 2024.

APPENDIX

A. Input Rotation MIMO

Figure 12 illustrates the **Input Rotation (In-Rot) MIMO** scheme, where the input of convolution, instead of the output of convolution, is rotated. For example, the inputs of the convolution, denoted as ciphertext $[v_1]_C$ and $[v_2]_C$, are rotated into $[v'_1]_C$ and $[v'_2]_C$. Subsequently, the rotated inputs are convolved with the kernel group $\{k_{12}, k_{21}\}$ and $\{k_{14}, k_{23}\}$ to directly obtain the intermediate ciphertext $[c_2k_{12}, c_1k_{21}]_C$ and $[c_4k_{14}, c_3k_{23}]_C$. In this way, combined with additional two intermediate ciphertext: $[c_1k_{11}, c_2k_{22}]_C$, $[c_3k_{13}, c_4k_{24}]_C$, the first output ciphertext, Output-1, is produced by summing up all four intermediate ciphertext using the Add operation. The process for obtaining the second output ciphertext, Output-2, follows a similar procedure. It's important to note that one Rotation operation is involved in converting the input ciphertext $[v_1]_C$ into $[v'_1]_C$. Following this, an additional $(k_w k_h - 1)$ in-Rot operations are performed over $[v'_1]_C$ to obtain the rotated ciphertexts necessary for the convolution (represented by the red $*$ convolution operator) between $[v'_1]_C$ and the kernel group $\{k_{12}, k_{21}\}$, as well as all other kernel groups to be convolved with $[v'_1]_C$. Obviously, all expensive Rotation operations are related to the kernel group marked in white. The proposed exRot-Free Pattern, which involves only the kernel group marked in gray, does not require any Rotation operations for calculation.

B. Block-Wise Performance Breakdown

Comparing StriaNet-L3 with ResNet-50 (see Figure 13), StriaNet consistently maintains significantly lower HE cost across all layers, particularly in deep layers, resulting in a 44.1% of ResNet-50's cost. Contrasting with the lightweight MobileNet (see Figure 14), StriaNet-L1 exhibits a notable advantage over MobileNet in the HE cost of shallow layers

TABLE XIII
STRIANET ARCHITECTURES FOR TINY IMAGENET.

Layers	Output Size	Cn	StriaNet-M1	StriaNet-M2
Conv	64×64		3×3, 32	
StriaBlock	64×64	2	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 32 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 32 \end{bmatrix} \times 3$
StriaBlock	32×32	8	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 64 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 64 \end{bmatrix} \times 4$
StriaBlock	16×16	32	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 96 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 128 \end{bmatrix} \times 6$
StriaBlock	8×8	128	$\begin{bmatrix} 1 \times 1, 1024 \\ 3 \times 3, 1024 \\ 1 \times 1, 128 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 1024 \\ 3 \times 3, 1024 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
Conv	8×8	128	1×1, 1024	
Classification	1×1		Avg pool	
			200D, FC, softmax	

due to superior dimension design aligned with unique HE characteristics. Additionally, despite higher accuracy, StriaNet requires fewer building blocks than MobileNet, i.e., 10 v.s. 17, resulting in an overall cost approximately 2/3 of MobileNet. This efficiency stems from HE-Efficient designs, allowing StriaNet to maintain a comparable Rotation complexity and HE cost as MobileNet while accommodating more effective parameters for enhanced capacity.

C. StriaNet on Tiny ImageNet and CIFAR-10 Classification

We extended our investigations using the Tiny ImageNet and CIFAR-10 datasets to evaluate StriaNet's performance with moderate to small-scale datasets. The network architectures are outlined in Table XIII and Table XIV. Our focus lies on understanding StriaNet's behavior across diverse dataset scales rather than aggressively pursuing state-of-the-art results. Nevertheless, these additional studies provide valuable insights into the adaptability and efficacy of StriaNet across varying dataset scales.

For the Tiny ImageNet dataset, shown in Table X and Figure 15, StriaNet is compared with the original ResNet-50, a pruned ResNet-50 using the state-of-the-art HE-friendly pruning method [62], VGG-16, pruned VGG-16 and MobileNetV2.

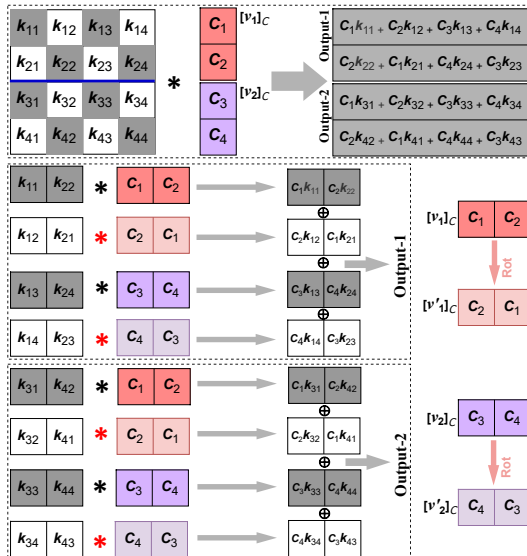


Fig. 12. In-Rot MIMO ($c_n = 2$).

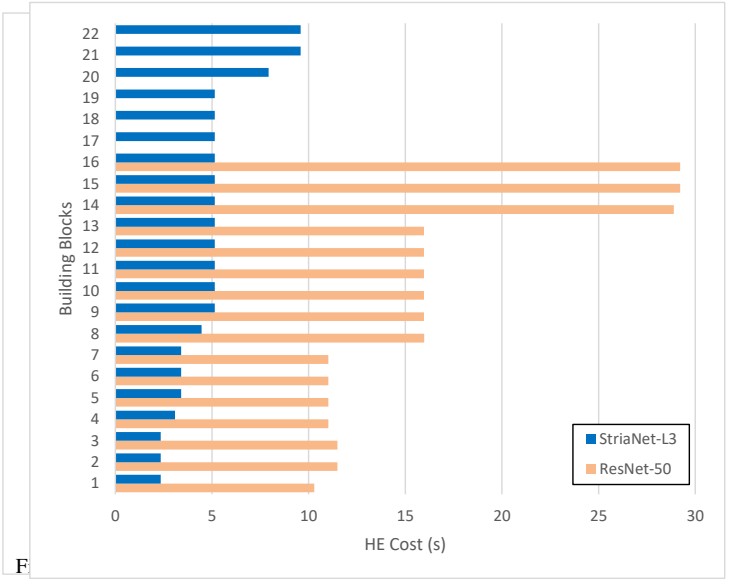


Fig. 13. L3 and ResNet.

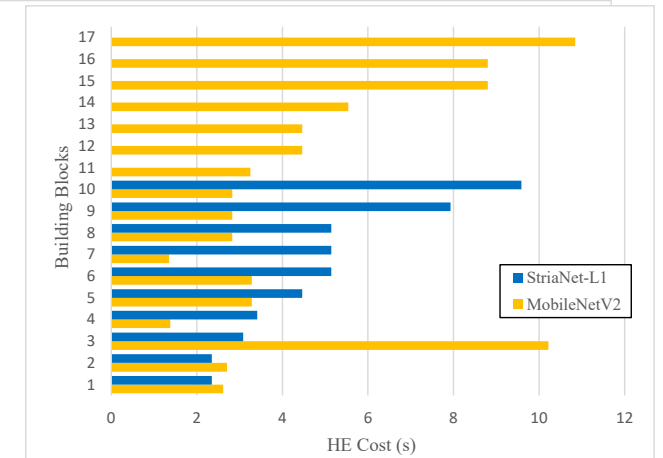


Fig. 14. Block-Wise Performance Breakdown: Comparison between StriaNet-L1 and MobileNet.

First, we introduce StriaNet-M2, achieved by simplifying the StriaNet-L Series. StriaNet-M2 demonstrates remarkable results, achieving the $3.05\times$, $1.27\times$, $3.87\times$, $1.16\times$ and $1.55\times$ speedup compared to ResNet-50, pruned ResNet-50, VGG-16, pruned VGG-16 and MobileNetV2, respectively, with higher accuracy of $2.83\% \uparrow$, $2.79\% \uparrow$, $3.57\% \uparrow$, $3.63\% \uparrow$ and $0.96\% \uparrow$. Additionally, through more aggressive compression, we introduce StriaNet-M1. Maintaining superior accuracy compared to the baseline, StriaNet-M1 achieves an increased

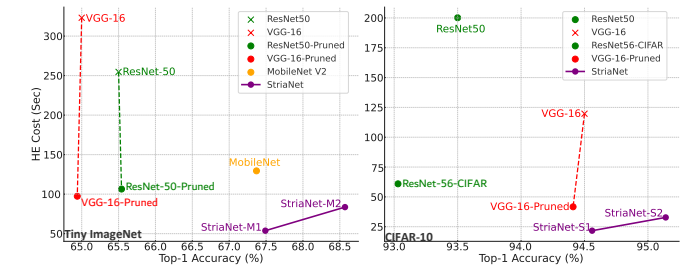


Fig. 15. Performance curve of StriaNet-M and StriaNet-S Series on Tiny ImageNet and CIFAR-10 Classification.

TABLE XIV
STRIANET ARCHITECTURES FOR CIFAR-10.

Layers	Output Size	Cn	StriaNet-S1	StriaNet-S2
Conv	32×32		3×3, 64	
StriaBlock	32×32	8	$\begin{bmatrix} 1\times 1, 128 \\ 3\times 3, 128 \\ 1\times 1, 64 \end{bmatrix} \times 1$	$\begin{bmatrix} 1\times 1, 128 \\ 3\times 3, 128 \\ 1\times 1, 64 \end{bmatrix} \times 2$
StriaBlock	16×16	32	$\begin{bmatrix} 1\times 1, 1024 \\ 3\times 3, 1024 \\ 1\times 1, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 1\times 1, 512 \\ 3\times 3, 512 \\ 1\times 1, 128 \end{bmatrix} \times 2$
StriaBlock	8×8	128	$\begin{bmatrix} 1\times 1, 2048 \\ 3\times 3, 2048 \\ 1\times 1, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 1\times 1, 1024 \\ 3\times 3, 1024 \\ 1\times 1, 128 \end{bmatrix} \times 2$
StriaBlock	4×4	512	—	$\begin{bmatrix} 1\times 1, 2048 \\ 3\times 3, 2048 \\ 1\times 1, 128 \end{bmatrix} \times 2$
Classification	1×1		Avg pool 10D, FC, softmax	

speedup of 4.74×, 1.98×, 6.01×, 1.81× and 2.41×.

For the CIFAR-10 dataset, shown in Table X and Figure 15, we compare StriaNet with the original ResNet-50, VGG-16, pruned VGG-16 and a CIFAR-10-specific architecture referred to as ResNet-56-CIFAR [21]. First, we introduce StriaNet-S2, achieved by further simplifying the StriaNet-M Series. StriaNet-S2 also demonstrates excellent performance, achieving a 6.1×, 3.65×, 1.27× and 1.63× speedup compared to ResNet-50, VGG-16, pruned VGG-16 and ResNet-56-CIFAR, respectively, with higher accuracy of 1.64% ↑, 0.64% ↑, 0.73% ↑ and 2.11% ↑. Additionally, with more aggressive compression, we introduce StriaNet-S1. With higher accuracy, StriaNet-S1 achieves an even higher 9.24×, 5.52×, 1.92× and 2.81× speedup.

D. Detailed Training Setup

General Setup		
·Pytorch-v1.12.1	·Optimizer: SGD	·Seed: int(2)
·Momentum: 0.9	·Weight decay: 5e-4	
StriaNet-Large on ImageNet		
·Learning rate: 0.01, decayed by 0.96 per epoch	·Batch size: 128	
·Training epochs: 200 + 50 (optional)		
StriaNet-Mid on Tiny ImageNet		
·Learning rate: 0.01, decayed by 0.1 per 100 epochs	·Batch size: 64	
·Training epochs: 120		
StriaNet-Small on CIFAR-10		
·Learning rate: 0.01, decayed by 0.1 per 100 epochs	·Batch size: 64	
·Training epochs: 250 + 50 (optional)		

We train StriaNet with the ImageNet, Tiny ImageNet, and CIFAR-10 dataset using the specified setup shown above. The DistributedDataParallel (DDP) module [75] of PyTorch is employed for training with distributed data parallelism to accelerate the training process. When training with multiple GPUs (e.g., N GPUs), the mini-batch size should be set as the actual batch size divided by N . We utilize SEED function to ensure the reproducibility of experimental results. However, disabling SEED can speed up the training process with minimal to no accuracy drop if reproducibility is not a concern.

Another observed situation is that when swapping between Windows and Linux OS with different versions of PyTorch, the trained model yields accuracy with very minor differences—some higher, some lower. This variation may be

attributed to differences in the random mechanisms between PyTorch versions. For fairness, we selected the lowest accuracy to represent in this paper.

The trained models and codes are available as attachments to this paper. The trained models include StriaNet-L1 to StriaNet-L4 for ImageNet, StriaNet-M1 and StriaNet-M2 for Tiny ImageNet, and StriaNet-S1 and StriaNet-S2 for CIFAR-10. All evaluations are provided through “*StriaNet_×_eval.py*” and a training code sample (StriaNet-L3, ImageNet, top-1 Acc: 76.33%) is available via “*Training_Sample_StriaNet_L3.py*”. The training adopts the “*DistributedDataParallel*” for multi-GPU acceleration, and it can be configured and conducted by the command:

“OMP_NUM_THREADS=2 torchrun -nproc_per_node=3 -nnodes=1 Training_Sample_StriaNet_L3.py”.